

Lab 07 - Global Descriptors, Template Matching and Brand-Consistency Scoring

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 03: AI-Based Visual Features, Styles and Consistency Outcome: ELO3 | In-class time: 30 minutes of scheduled practical time

Competency	Statement
Ability A3	Analyse global feature descriptions
Ability A4	Design and implement feature extraction and representation methods
Knowledge K7	Global feature descriptions, statistical and geometrical methods

Scenario

Twelve assets have come back from three different creators for the same campaign. Three contain deliberate fixture deviations. Build a global-descriptor fingerprint - colour histogram, saturation, edge density and Hu moments - score every asset against the approved master, and use template matching to verify the logo is present and correctly placed.

What you will produce

out/consistency-scores.csv ranking all 12 assets against the master with four global descriptors and a combined score, plus out/logo-placement.csv from template matching, and a reviewed list of fixture deviations with the descriptor or logo gate that flagged each, including any misses.

Tools: Python 3 | opencv-python | NumPy | no network, no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are `*not*` in that wheel and are deliberately avoided: `cv2.TrackerCSRT_create` / `cv2.TrackerKCF_create` and the `cv2.quality SSIM` module, both of which live in `opencv-contrib-python`. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Fingerprint the approved master

- Score 12 assets on four global descriptors
- Combine into one consistency score
- Template-match the logo and check placement
- Name the outliers and the descriptor that caught each

Files in this folder

Path	What it is
reference/master/nq-master.png	The approved campaign master, 900x700 (deterministically rendered - SIMULATED)
reference/assets/asset_01.png .. asset_12.png	Twelve delivered assets, nine on-brand and three deliberately off-brand in different ways - one recoloured, one over-sharpened, one with the logo misplaced (deterministically rendered - SIMULATED)
reference/nq-logo-template.png	The 120x60 logo template used for matching (deterministically rendered - SIMULATED)
data/consistency-thresholds.json	Per-descriptor pass thresholds and the combined-score weights
data/asset-manifest.csv	The twelve assets with their creator, delivery date and declared variant
consistency.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

- Read data/asset-manifest.csv. Twelve assets, three creators. Do NOT look at the images first. Rank numerical deviations without inferring quality from creator identity; then inspect the flagged assets and record false positives or missed defects.
- Fingerprint the master. Descriptor 1 is the normalised 2-D hue-saturation histogram, 30 hue bins x 32 saturation bins, computed with `cv2.calcHist` and normalised with `cv2.normalize(..., 0, 1, cv2.NORM_MINMAX)`. Normalisation is what makes the comparison size-independent.

```
hsv = cv2.cvtColor(master, cv2.COLOR_BGR2HSV)
hist = cv2.calcHist([hsv], [0, 1], None, [30, 32], [0, 180, 0, 256])
cv2.normalize(hist, hist, 0, 1, cv2.NORM_MINMAX)
```

- Compare a histogram with itself using all four `cv2.compareHist` methods and record the self-comparison values: CORREL gives 1.0, CHISQR gives 0.0, INTERSECT gives the histogram sum, BHATTACHARYYA gives 0.0. Knowing which direction 'good' points in for each method prevents an inverted gate.

```
for name, m in [('CORREL', cv2.HISTCMP_CORREL), ('CHISQR', cv2.HISTCMP_CHISQR),
               ('INTERSECT', cv2.HISTCMP_INTERSECT),
               ('BHATTACHARYYA', cv2.HISTCMP_BHATTACHARYYA)]:
    print(name, cv2.compareHist(hist, hist, m))
```

4. Use BHATTACHARYYA for the gate: it is bounded in [0,1], 0 means identical, and it behaves well on sparse histograms. Score all twelve assets against the master and record the distance.
5. Descriptor 2 - mean HSV saturation. This single number can flag saturation changes, but cannot detect hue-only recolouring. Compute it for the master and for all twelve, and record the absolute difference.
6. Descriptor 3 - Canny edge density, using the thresholds you settled on in Lab 06. This can flag texture/edge changes; it cannot uniquely identify sharpening or blur. Record the ratio $\text{asset_density} / \text{master_density}$ rather than the raw difference, so the threshold is dimensionless; compare equal image sizes because edge density remains resolution-sensitive.
7. Descriptor 4 - Hu moments. These are seven values computed from normalised central moments with ideal translation, scale and rotation invariance; rasterisation and segmentation affect them. Otsu gives a global brightness mask, not a verified subject silhouette. Inspect that mask before interpreting shape similarity, and compare with `cv2.matchShapes`.

```
_, binimg = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
hu = cv2.HuMoments(cv2.moments(binimg)).flatten()
print(cv2.matchShapes(master_bin, binimg, cv2.CONTOURS_MATCH_I1, 0.0))
```

8. Take the log of the Hu moments before you compare them by hand: raw Hu values span many orders of magnitude, so $-\text{sign}(h) * \log_{10}(|h|)$ is the standard readable form. Record the seven log values for the master.
9. Combine the four descriptors into a single consistency score using the weights in `data/consistency-thresholds.json`. Normalise each descriptor to [0,1] first - you cannot weight a Bhattacharyya distance against a saturation difference in raw units.

```
python3 consistency.py --master reference/master/nq-master.png --assets reference/assets --
thresholds data/consistency-thresholds.json --out out/consistency-scores.csv
```

10. Read `out/consistency-scores.csv` sorted by combined score. Three fixture assets were deliberately modified, but need not all cross this numerical threshold. Join descriptor and logo-gate rejects by asset name, inspect them, and explain misses.
11. Inspect the ranked images and compare numeric flags with visible defects. Record false positives and false negatives without blaming a creator from metadata alone.
12. Now template matching. Load `reference/nq-logo-template.png` and run `cv2.matchTemplate(asset, template, cv2.TM_CCoeff_NORMED)` for each asset. The result is a response map, not a box: every position gets a similarity score.

```
res = cv2.matchTemplate(asset_gray, tpl_gray, cv2.TM_CCoeff_NORMED)
minv, maxv, minl, maxl = cv2.minMaxLoc(res)
print('best score %.3f at %s' % (maxv, maxl))
```

13. Note the two methods where LOW is best. For `TM_SQDIFF` and `TM_SQDIFF_NORMED` you take `minLoc`; for the other four you take `maxLoc`. Getting this backwards puts your detection in the opposite corner of the frame.
14. Apply the score threshold from `data/consistency-thresholds.json`. Report, per asset: the peak score, the peak location, whether it exceeds the threshold, and whether the top-left location falls inside the required placement box. This box constrains the logo origin, not full-box containment.
15. State template matching's limits explicitly in your write-up: it is not scale- or rotation-invariant, and it degrades under illumination change unless you use a normalised method. That is exactly why Lab 06's ORB matching exists alongside it.
16. Save `out/consistency-scores.csv` and `out/logo-placement.csv`, name the three fixture deviations with the descriptor or logo gate that caught each (or explain a miss), and complete the evidence checklist.

Verify

out/consistency-scores.csv ranks all 12 assets with threshold-based decisions, each flag traceable to its weighted descriptor contribution, and out/logo-placement.csv shows one asset whose logo peak score passes but whose peak location falls outside the required placement box.

Expected outputs

- out/consistency-scores.csv - 12 rows: asset, hist_bhattacharyya, saturation_delta, edge_density_ratio, shape_match, combined_score, verdict.
- Join both CSV gate verdicts by asset and compare flagged assets against all three fixture defects; report misses honestly.
- Inspect whether histogram distance flags recolouring; saturation delta only changes if saturation changes.
- Inspect whether edge density flags the modified texture; do not infer a sharpening cause from density alone.
- out/logo-placement.csv - 12 rows: peak score, peak (x,y), score_pass, placement_pass.
- The misplaced-logo asset passing on score and failing on placement.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- [] out/consistency-scores.csv sorted by combined score.
- [] The four self-comparison values for the histogram methods, showing which direction is 'good'.
- [] The seven log-Hu moments of the master.
- [] All three named outliers, each with the mechanism that caught it: two by a named descriptor, one by the logo placement gate.
- [] out/logo-placement.csv showing the score-pass / placement-fail asset.
- [] One sentence stating the two template-matching methods where minLoc is used.
- [] One sentence stating template matching's invariance limits.

Troubleshooting

Symptom	What to do
Every asset scores as an outlier	You compared un-normalised histograms, or mixed a normalised master against un-normalised assets. Normalise every histogram with the same call before comparing.
cv2.compareHist raises a type error	Both histograms must be float32 of the same shape. cv2.calcHist returns float32; do not cast to float64 in between.
Hu moments are all zero	The binary mask is empty. Check the Otsu threshold actually split the image - on a low-contrast asset, threshold the saturation channel instead of the grayscale.
matchShapes returns a huge number	One of the two inputs has no contour. Pass a binary image with a real foreground, inspect the segmentation and note that CONTOURS_MATCH_I1 is not bounded to [0,1].
Template match peaks in the wrong corner	You used minLoc with a CCoeff/CCorr method, or maxLoc with a SQDIFF method. SQDIFF and SQDIFF_NORMED use minLoc; everything else uses maxLoc.

Symptom	What to do
Template match score is low on an asset you know contains the logo	The logo has been scaled or rotated in that delivery. Template matching cannot absorb either - record it as a limitation finding and match with ORB instead.

References used in this lab

- [S9] Tschochohei and Schenker (2026), as above - *Ch.5 pp.76-77 and pp.82-83*.

Natively multimodal LLMs reason over text and images in one model, which is what makes character/product consistency across images tractable; enterprise deployment practices - brand style guide for AI, creative-workflow integration, review and approval gates, centralised asset management, prompt-engineering literacy.

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD², LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.